
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

We introduce FLOWTIME, an autoregressive flow-matching model for probabilistic forecasting of multivariate timeseries. The central contribution is a forecasting factorization that turns the joint distribution over future trajectories into a sequence of conditional next-step distributions, each learned by a shared flow conditioned on past observations and optional future covariates. This design keeps the scientific objective explicit: sample calibrated future trajectories while avoiding direct modeling of a fixed high-dimensional forecast window. By coupling this autoregressive decomposition with the simulation-free flow matching (FM) objective, FLOWTIME combines the extrapolation and compactness advantages of autoregressive forecasters with the ability of transport-based generative models to represent complex conditional distributions. We evaluate this contribution on stochastic dynamical systems and real-world forecasting datasets, showing that the same method improves or remains competitive across prediction, extrapolation, and uncertainty-quantification settings.

1 INTRODUCTION

Probabilistic timeseries forecasting asks for more than a single future trajectory: it requires a predictive distribution that extrapolates system dynamics from observed data. This paper targets that setting because calibrated trajectory-level uncertainty is essential across scientific, industrial, and societal forecasting applications (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024).

Deep generative forecasting methods provide a natural route to this goal by treating prediction as conditional sampling from a future probability density (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021). Diffusion and score-based models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020) have therefore become strong baselines on forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kolloviah et al., 2023; Meijer & Chen, 2024). Their contribution to the field is clear, but their iterative stochastic denoising procedures often impose substantial training and inference costs.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) offers the modeling ingredient we build on. Instead of learning a discretized stochastic denoising chain, FM learns deterministic probability paths that transform an arbitrary base distribution into a target distribution through continuous normalizing flows. This framework removes handcrafted noise schedules and long ancestral sampling chains, making it a useful foundation for a forecasting method whose contribution depends on both expressivity and efficient conditional sampling.

Existing FM forecasting methods (Tamir et al., 2024; Hu et al., 2024; Kolloviah et al., 2024) show that this framework can be competitive and more computationally efficient than diffusion-based alternatives. The remaining gap, and the motivation for this paper, is their fixed-window formulation: they learn the conditional distribution of an entire future horizon given a fixed context horizon. That formulation can train and sample quickly, but it also makes extrapolation beyond the training horizon difficult, can yield miscalibrated uncertainty, and forces the model to learn complicated time-dependent probability paths along the flow dimension. Prior work addresses this optimization difficulty with Gaussian-process-informed priors (Kolloviah et al., 2024), but that design restricts the approach to univariate forecasting.

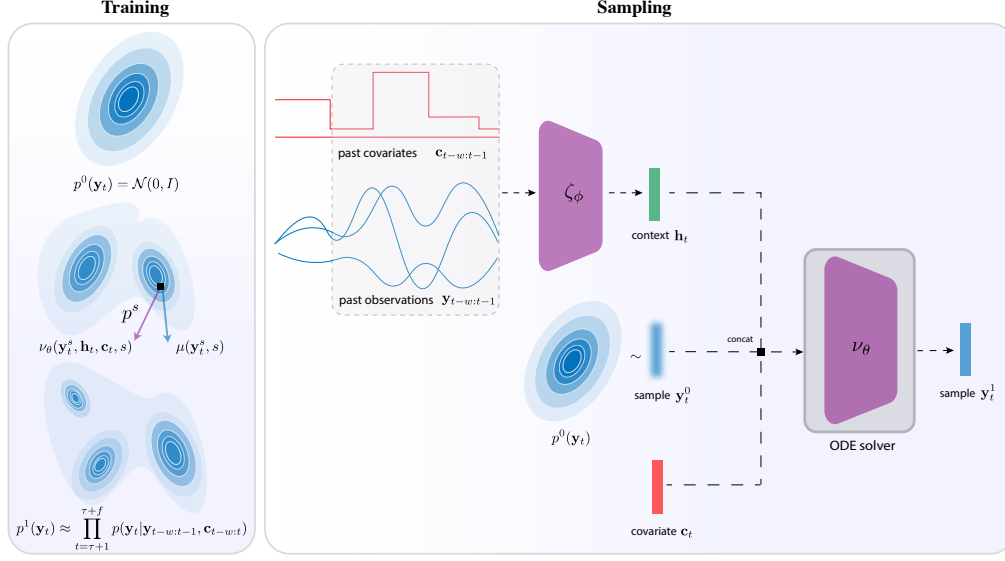


Figure 1: Overview of FLOWTIME’s contribution: each forecast step is learned as a conditional flow-matching problem and reused autoregressively at sampling time. During training, a probability path $(p^s)^{0 \leq s \leq 1}$ connects a base distribution p^0 to the target distribution p^1 . A velocity field μ generates this path, and a neural network ν_θ learns it from a path sample y_t^s , the flow step s , a context vector h_t encoding past observations and covariates, and the current covariate c_t . At inference, the model samples from p^0 and integrates the learned velocity field with an ODE solver until $s = 1$, then feeds the sampled observation into the next autoregressive context.

Our main methodological contribution is FLOWTIME: a simple FM forecaster that scales to multivariate datasets without requiring informed priors. Rather than model the future window all at once, FLOWTIME decomposes forecasting into conditional next-step distributions over future time points, with each conditional distribution learned by a shared flow from a simple base distribution to the next-observation distribution. This autoregressive flow-matching structure front-loads the advantages we seek: variable forecast horizons, extrapolation beyond the training window, explicit propagation of uncertainty through time, and a simpler conditional learning problem.

Empirically, we support this contribution in two complementary settings. On stochastic dynamical systems, we isolate the value of the autoregressive factorization against a non-autoregressive FM baseline; on real-world datasets, we compare FLOWTIME with statistical, deep autoregressive, diffusion, and flow-matching baselines.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) supplies the generative modeling foundation for our contribution. It trains continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018) at large scale, while CNFs themselves represent data transformations as ordinary differential equations (ODEs) that induce invertible mappings with tractable likelihoods.

Like CNFs, FM learns a time-dependent diffeomorphic map on the data space Ω , denoted by the flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. The role of this flow in our paper is direct: it provides the mechanism that converts a simple source draw into a conditional forecast draw. In the general FM setting, a sample $\mathcal{X}^0 \sim p$ from source distribution p is transported to $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ with $\mathcal{X}^1 \sim q$ for target distribution q . The map is constructed by solving the following initial value problem:

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

Here $\mu: [0, 1] \times \Omega \rightarrow \Omega$ is the vector field that sets the flow velocity and induces a probability path $(p^s)^{0 \leq s \leq 1}$ over Ω , with $p^0 = p$. Learning FM therefore means learning a vector field μ whose endpoint distribution satisfies $p^1 = q$.

The practical reason we use FM is that it avoids the CNF requirement of solving and differentiating through Eq. (1) during likelihood maximization. FM instead learns a parameterized vector field ν_θ by regression against vector fields of fixed conditional probability paths. This yields the tractable conditional flow matching (CFM) objective:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0,1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

After minimizing this objective, samples from q are obtained by solving Eq. (1) with the trained ν_θ . This simulation-free training objective is the background contribution we reuse for forecasting: it avoids backpropagation through the ODE solver while retaining scalable generative modeling behavior in high-dimensional domains such as image and video modeling (Esser et al., 2024; Polyak et al., 2024). Existing path designs and conditioning strategies further show that FM unifies several transport-based generative models (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023).

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting We frame the method around the object reviewers should track throughout the paper: a sampler for the conditional future trajectory distribution. Let $\mathbf{y}_\tau \in \mathbb{R}^n$ be an n -dimensional observation at time τ . Given l past measurements including the current observation, $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, we seek probabilistic forecasts of the next f values, $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$. Forecasts may also condition on observed covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$ with $\mathbf{c}_\tau \in \mathbb{R}^c$. Formally, using $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, the goal is to sample from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$.

The first method contribution is to replace a single fixed-horizon joint model with an autoregressive Markov factorization of order w :

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ is the history size. This shifts the learning target from the full future-window distribution to the reusable conditional distribution in Eq. (3). Rather than fitting those conditionals by maximum likelihood, as in many autoregressive models, we learn them with flow matching (Lipman et al., 2022).

This factorization is the main structural contribution of FLOWTIME. It reduces the modeling task to simpler conditionals $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ rather than a high-dimensional joint future distribution; it permits teacher-forced training of each conditional while retaining parallelism across time points; it exposes the window size w as a capacity–cost trade-off; and it makes uncertainty propagation explicit because forecast samples move through the same chain of conditional distributions used to define the model.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For each future time step $t \in [\tau + 1, \tau + f]$, the second method contribution is to attach a flow-matching probability path to the autoregressive conditional. We define $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$ with marginals $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$, where $p_{\mathcal{D}}$ is the empirical distribution. A flow field $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ generates this path along s through the ordinary differential equation (ODE):

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the vector field used to construct a flow satisfying the continuity equation.

To keep this autoregressive conditional flow trainable, we express the marginal probability path as a mixture of conditional probability paths:

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t|\mathbf{z})p(\mathbf{z})d\mathbf{z} \quad (5)$$

Here $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is a conditioning random variable drawn from an arbitrary data coupling $\pi_{0,1}$. The relevance for FLOWTIME is that Theorem 2 in (Lipman et al., 2022) lets us train on conditional

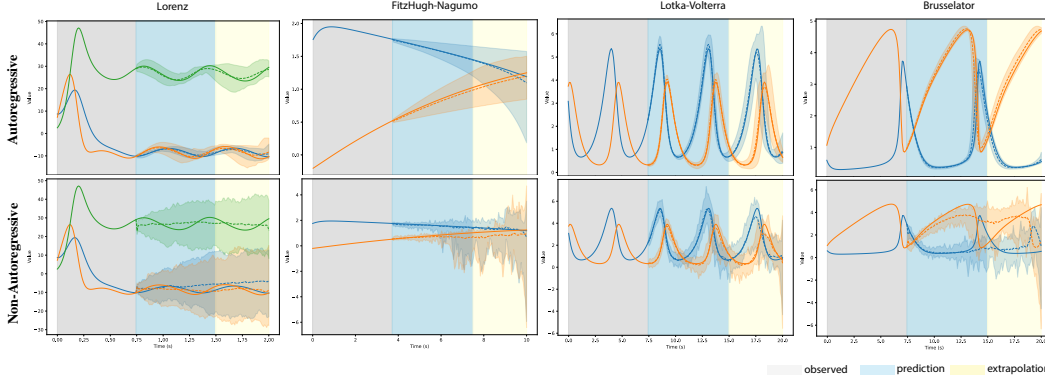


Figure 2: Qualitative comparison highlighting the contribution of autoregressive flow matching relative to standard flow matching on four stochastic dynamical systems. Solid lines show ground truth, and dashed lines show the mean prediction with the 95% confidence interval. Each panel separates the trained prediction regime from the extrapolation regime, making the effect of the autoregressive factorization visible beyond the training window.

velocity fields $\mu(\mathbf{y}_t, s|\mathbf{z})$ for paths $p^s(\mathbf{y}_t|\mathbf{z})$ while recovering the corresponding marginal velocity field $\mu(\mathbf{y}_t, s)$.

Following (Tong et al., 2023), we use the simple coupling $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are samples from p^0 and p^1 . This yields the conditional probability path:

$$p^s(\mathbf{y}_t|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

For $\sigma^2 \rightarrow 0$, the marginals satisfy $p^0(\mathbf{y}_t|\mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t|\mathbf{z}) = p^1(\mathbf{y}_t)$. The corresponding conditional velocity field is simply $\mu(\mathbf{y}_t, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This straight Euclidean path is deliberately simple: it gives closed-form targets for efficient training and sampling. Once the velocity field is learned, we sample the target conditional $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ by solving the ODE in Eq. (4) until $s = 1$.

3.2 TRAINING

Training makes the two contributions operational: the flow learns a next-step conditional, and the context encoder supplies the autoregressive history. As in standard FM, we regress the velocity field onto a neural network ν_θ with parameters θ using the conditional flow matching objective in Eq. 2. For the forecasting context, we define $\mathbf{h}_t \in \mathbb{R}^h$ to encode the time- t context window:

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a neural network with parameters ϕ . The resulting learning objective is:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t|\mathbf{z})} \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

We optimize this objective by sampling a time point t , its context window, and covariates from $\mathcal{D}$, then jointly updating ϕ and θ by stochastic gradient descent on $\mathcal{L}(\theta, \phi)$. This is a teacher-forced training regime (Williams & Zipser, 1989): ground-truth histories, rather than model-generated histories, condition each training example. The result is a reviewer-visible trade-off in the method: training remains parallel across time steps, while sampling later exposes the intended autoregressive uncertainty propagation. Algorithm 1 summarizes this training procedure.

3.3 INFERENCE

At inference time, the learned contribution is used as a trajectory sampler for new observations $\mathbf{Y}_t$ and covariates $\mathbf{C}$ over f future timestamps. Algorithm 2 applies the learned conditional flow autoregressively with a rolling window: each sampled future observation is appended to the context before the next step is generated. Repeating this sampling procedure produces empirical predictive quantiles and exposes uncertainty propagation across the forecast horizon.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
- 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
- 5: Compute target: $\mu(\mathbf{y}_t^s, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
- 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
- 7: Update θ, ϕ
- 8: **end while**
- 9: **return** ν_θ, ζ_ϕ

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
- 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
- 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
- 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
- 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s)$, $\psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0$, $s \in [0, 1]$
- 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
- 7: **end for**
- 8: **return** $\mathbf{Y}_f$

4 EXPERIMENTS

The experiments are organized to make the paper’s contributions separately visible. First, simulated stochastic dynamical systems isolate the effect of the autoregressive FM factorization against a non-autoregressive FM ablation. Second, real-world datasets test whether the same contribution remains competitive against established statistical, deep generative, diffusion, and flow-matching forecasters.

4.1 DYNAMICAL SYSTEMS

Data The dynamical-system experiment tests the claimed extrapolation and uncertainty-propagation benefits in a controlled setting. We evaluated FLOWTIME on stochastic versions of five classical systems: the Lorenz system (Lorenz, 1963), FitzHugh–Nagumo model (FitzHugh, 1961), Lotka–Volterra system Lotka (1925), Brusselator (Lefever & Prigogine, 1968), and Van der Pol oscillator (van der Pol, 1926). For each system, trajectories were generated by numerically solving the corresponding stochastic differential equations from randomly sampled initial conditions and Brownian motion realizations. Appendix A.1 gives the full data-generation details.

Implementation details To keep this ablation focused on factorization rather than architecture scale, we used compact networks. The context encoder ζ_ϕ was a two-layer bi-directional LSTM with 64 hidden units, and the velocity network ν_θ was an MLP with three hidden layers of size 64. Fourier positional embeddings encoded the flow step s as a 16-dimensional vector with fixed frequencies, and the context length w was set equal to the prediction length. Models were trained by stochastic gradient descent with Adam, learning rate 0.003, and batch size 128; convergence took ~ 7 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines The comparison is designed as a direct contribution test: does the autoregressive factorization improve an otherwise related FM forecaster? We therefore compare FLOWTIME with a non-autoregressive flow-matching baseline that models the full future joint distribution conditioned on past observations and covariates, $p^1 = p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. This baseline isolates the effect of the factorization introduced in FLOWTIME and closely resembles the method of (Kolloviev et al., 2024). Appendix B provides additional details.

Evaluation The evaluation separates ordinary prediction from the more reviewer-relevant extrapolation claim. Each trajectory was partitioned into (1) an observation window used as historical input, (2) a prediction window available during training as the target, and (3) an extrapolation window that remained unseen during training. We report the continuous rank probability score (CRPS) (Winkler et al., 1996) to measure probabilistic forecast quality and the normalized root mean square error (NRMSE) (Hyndman & Koehler, 2006) to measure point-prediction accuracy.

Table 1: Ablation of the paper’s central structural contribution on five stochastic dynamical systems: autoregressive FLOWTIME versus a non-autoregressive (Non-AR) flow-matching factorization. CRPS and NRMSE are reported on the test set for both the trained prediction regime and the unseen extrapolation regime. Values are means and standard deviations across five random seeds.

System	Factorization	Prediction		Extrapolation	
		CRPS ↓	NRMSE ↓	CRPS ↓	NRMSE ↓
Lorenz	Non-AR	0.147±0.012	0.242±0.018	0.120±0.009	0.301±0.024
	FLOWTIME	0.090±0.008	0.017±0.003	0.137±0.011	0.048±0.006
FitzHugh-Nagumo	Non-AR	0.165±0.014	0.131±0.011	0.214±0.016	0.306±0.022
	FLOWTIME	0.144±0.013	0.090±0.007	0.146±0.012	0.278±0.019
Lotka-Volterra	Non-AR	0.130±0.011	0.266±0.019	0.136±0.012	0.502±0.033
	FLOWTIME	0.126±0.010	0.189±0.015	0.123±0.009	0.339±0.026
Brusselator	Non-AR	0.217±0.016	0.658±0.042	0.238±0.019	1.046±0.057
	FLOWTIME	0.125±0.011	0.031±0.004	0.109±0.008	0.023±0.003
Van der Pol	Non-AR	0.143±0.012	0.161±0.014	0.211±0.018	0.510±0.031
	FLOWTIME	0.225±0.017	0.051±0.005	0.282±0.021	0.101±0.009

Results Table 1 supports the contribution claim that the autoregressive factorization improves the FM forecaster on dynamical systems. In the prediction window, FLOWTIME achieves lower NRMSE than the non-AR baseline on all five systems, with the largest reductions on Lorenz (0.017 vs 0.242) and Brusselator (0.031 vs 0.658), corresponding to 93% and 95% lower error. The CRPS comparison also favors the autoregressive model on four of five systems, with Van der Pol as the exception where the non-AR baseline has lower CRPS. The extrapolation window is the sharper test of the proposed structure: FLOWTIME preserves its NRMSE advantage on every system, including a 98% reduction on Brusselator (0.023 vs 1.046). These results indicate that explicitly modeling temporal dependencies through a chain of conditional flows improves accuracy and often improves probabilistic forecast quality, particularly when forecasts extend beyond the training regime.

4.2 REAL-WORLD DATASETS

Data The real-world benchmark tests whether the same autoregressive FM contribution transfers beyond simulated dynamics. We evaluated FLOWTIME on univariate and multivariate GluonTS forecasting tasks (Alexandrov et al., 2020) with hourly and daily frequencies: Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019). Appendix A.2 reports additional dataset details.

Implementation details For the real-world datasets, we used a stronger architecture while preserving the same contribution structure. The encoder ζ_ϕ was a three-layer bi-directional LSTM with 64 hidden units. The velocity network ν_θ was a residual neural network with five 1-D convolutional residual blocks and gated activations (Van Den Oord et al., 2016; Kong et al., 2020), similar to (Rasul et al., 2021) but with fewer residual blocks. Fourier positional embeddings encoded the flow step s into a 32-dimensional vector, and the same embedding approach encoded time-dependent covariates c . Models were trained with Adam using learning rate 0.001 and batch size 128; depending on the dataset, convergence took 40 – 100 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines To place the contribution in the forecasting literature, we compare FLOWTIME with statistical, deep autoregressive, diffusion, and flow-matching baselines. The statistical baselines are Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008); the deep learning baselines include DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016). We also include diffusion baselines TSDiff (Kollovich et al., 2023), SSSD (Alcaraz & Strodthoff, 2022), and TimeGrad (Rasul et al., 2021), along with the flow-matching baseline TSFlow (Kollovich et al., 2024).

Table 2: Real-world forecasting comparison showing where the proposed autoregressive flow-matching model is strongest. Test-set CRPS is reported for five datasets, with lower values indicating better probabilistic forecasts. FLOWTIME means and standard errors are obtained by retraining and evaluating five times; baseline results are from (Kollovich et al., 2024). Best scores are in **bold**, and second-best scores are underlined.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060±0.000	0.013±0.000	0.512±0.000	0.221±0.000	0.423±0.000
ARIMA	0.344±0.000	0.013±0.000	0.558±0.003	0.486±0.000	0.654±0.000
ETS	0.056±0.000	0.008±0.000	0.550±0.000	0.492±0.000	0.651±0.000
DeepAR	0.051±0.000	0.013±0.004	0.429±0.015	0.103±0.000	0.215±0.003
WaveNet	0.058±0.008	0.012±0.001	0.360±0.000	0.099±0.000	0.207 ±0.000
CSDI	0.051±0.000	0.013±0.000	0.360±0.000	0.152±0.000	0.318±0.012
SSSD	0.048±0.000	0.010±0.000	0.354±0.024	0.107±0.002	0.209±0.000
TSFlow	0.045±0.001	0.009 ±0.001	0.343±0.002	0.083 ±0.000	0.227±0.000
FLOWTIME	0.042 ±0.001	0.009 ±0.001	0.284 ±0.002	<u>0.089</u> ±0.000	0.243±0.002

Results Table 2 shows that the proposed structure remains competitive on heterogeneous real-world data. FLOWTIME obtains the best CRPS on Electricity, Exchange, and Solar. On Electricity, its CRPS of 0.042 improves over the second-best TSFlow score of 0.045 by 6.7%. On Exchange, it matches TSFlow at 0.009 and outperforms the other listed statistical and deep learning methods. The largest real-world gain appears on Solar, where FLOWTIME reaches 0.284 CRPS, a 17.2% reduction relative to TSFlow at 0.343. The method ranks second on Traffic with 0.089, behind TSFlow at 0.083 but ahead of the remaining baselines, and remains competitive on Wikipedia with 0.243, behind WaveNet (0.207) and SSSD (0.209). Overall, the real-world results support the claim that autoregressive flow matching is not only an ablation improvement on simulated systems but also a strong general-purpose probabilistic forecasting design.

5 DISCUSSION

FLOWTIME’s contribution is the combination of an autoregressive forecasting factorization with a flow-matching conditional sampler. This design keeps temporal dependence explicit while retaining the expressivity of flow-based generative modeling, yielding a model for multivariate probabilistic forecasting rather than a fixed-horizon window generator. The experiments support this contribution in two ways. On simulated dynamical systems, the autoregressive decomposition improves accuracy and often probabilistic quality relative to a non-autoregressive FM baseline, especially in extrapolation windows. On real-world datasets, FLOWTIME is best or tied for best on three of five CRPS benchmarks and remains competitive on the others, showing that the same structural choice transfers beyond the controlled ablation setting.

The same structure also defines the main limitations. Because sampling is autoregressive, inference is sequential and can be more expensive than fully parallel non-autoregressive generation, which may matter in real-time forecasting systems. In addition, the present implementation performs flow matching directly in data space, a choice that may be suboptimal for high-dimensional or partially observed data. These limitations suggest natural extensions: latent-space flow matching, irregularly sampled observations, and broader scientific or decision-making applications where calibrated uncertainty is central. Within these boundaries, FLOWTIME shows that flow-based generative models can improve probabilistic forecasting when their contribution is structured around temporal conditionality rather than fixed-window generation.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
- Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>, 7(1):62, 2017.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.
- Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.
- Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–1910. PMLR, 2019.
- Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-Form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen. Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL <http://arxiv.org/abs/2411.07506>.
- Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.
- Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep variational Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint arXiv:1605.06432*, 2016.
- Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang, and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for probabilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–28364, 2023.

432 Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow
433 matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth*
434 *International Conference on Learning Representations*, 2024.

435 Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile
436 diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.

437 Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term
438 temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference*
439 *on Research & Development in Information Retrieval*, pp. 95–104, 2018.

440 R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal*
441 *of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.

442 Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical*
443 *Transactions of the Royal Society A*, 379(2194):20200209, 2021.

444 Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow match-
445 ing for generative modeling. In *The Eleventh International Conference on Learning Representa-*
446 *tions*, 2022.

447 Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data
448 with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.

449 Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.

450 Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.

451 Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv*
452 *preprint arXiv:2401.03006*, 2024.

453 Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv
454 Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media founda-
455 tion models. *arXiv e-prints*, pp. arXiv–2410, 2024.

456 Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Mul-
457 tivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint*
458 *arXiv:2002.06103*, 2020.

459 Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising dif-
460 fusion models for multivariate probabilistic time series forecasting. In *International Conference*
461 *on Machine Learning*, pp. 8857–8868. PMLR, 2021.

462 David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus.
463 High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances*
464 *in Neural Information Processing Systems*, 32, 2019.

465 David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic
466 forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):
467 1181–1191, 2020.

468 Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised
469 learning using nonequilibrium thermodynamics. In *International Conference on Machine Learn-*
470 *ing*, pp. 2256–2265. PMLR, 2015.

471 Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben
472 Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint*
473 *arXiv:2011.13456*, 2020.

474 Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow match-
475 ing for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Gener-*
476 *ative Modeling*, 2024.

486 Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based
487 diffusion models for probabilistic time series imputation. *Advances in Neural Information Pro-*
488 *cessing Systems*, 34:24804–24816, 2021.

489 Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguët, Yanlei Zhang, Jarriid Rector-
490 Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models
491 with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

492 Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves,
493 Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for
494 raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.

495 B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical*
496 *Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.

497 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
498 neural networks. *Neural Computation*, 1(2):270–280, 1989.

499 Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B
500 Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring
501 rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.

502 Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time
503 series foundation models: Generalizing time series representation with large language model. In
504 *ArXiv:2104.00164*, 2024.

505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539

A DATASETS

A.1 DYNAMICAL SYSTEMS

This appendix records the controlled data used to test the paper’s extrapolation and factorization claims. We generated training and testing data for five classical stochastic dynamical systems—Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator, and Van der Pol—by numerically integrating their corresponding stochastic differential equations (SDEs). Each system has the general form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \Theta) dt + \sigma d\mathbf{W}(t),$$

where $\mathbf{f}$ is the deterministic drift, Θ are the system parameters, σ is a constant diffusion term, and $\mathbf{W}(t)$ is a standard Brownian motion. For each system, we sampled 2400 random initial conditions from the uniform range in Table 3 and solved the SDE with the Euler-Heun method at fixed step size. Each trajectory contains 200 equally spaced time points, split to match the main experimental contribution: the first 75 points form the observation window, the next 75 form the prediction window, and the final 50 form the extrapolation window. We used 2000 trajectories for training and 400 for testing.

Table 3: Stochastic dynamical systems used for the controlled factorization and extrapolation experiments. $\mathbf{f}(\mathbf{x})$ denotes the drift function, Θ the system parameters, σ the constant diffusion term, and $p(\mathbf{x}(0))$ the initial condition distribution. Each system was integrated over $[t_0, t_1]$ with 200 Euler-Heun steps.

System	$\mathbf{f}(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh-Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka-Volterra	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\beta = 0.9$			
		$\gamma = 0.8$			
Brusselator	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$\delta = 1.8$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_2 = B x_1 - x_1^2 x_2$	$A = 1.0$			
		$B = 3.0$			
Van der Pol	$f_1 = x_2$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$				

A.2 REAL-WORLD DATASETS

The real-world experiments use five GluonTS datasets (Alexandrov et al., 2020) to evaluate FLOW-TIME beyond the controlled dynamical-system setting. We use the preprocessed versions from (Salinas et al., 2019), and Table 4 summarizes the dataset properties relevant to forecasting scale and horizon.

Table 4: Real-world dataset properties used to evaluate whether the proposed autoregressive flow-matching contribution transfers across dimensions, domains, frequencies, training lengths, and prediction horizons.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting This appendix specifies the ablation baseline used to isolate the contribution of FLOWTIME’s autoregressive factorization. The forecasting setup is the same as in Section 3: given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$, the model targets $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. The baseline removes the autoregressive decomposition and directly models the full future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation preserves temporal correlations across all future time steps, but it tests the cost of doing so by requiring a single high-dimensional distribution over the entire horizon.

Training For the non-autoregressive baseline, the probability path acts on the whole future window. We construct $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ to transport samples from $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ to $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$, where Σ is block diagonal. The prior can be interpreted as independent Brownian motion processes per dimension, with Brownian motion defined as $W: [\tau + 1, \tau + f] \rightarrow \mathbb{R}^n$.

The conditional path is a linear interpolation bridge with Brownian noise:

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$, $\mathbf{Y}_f^0 \sim p^0$, and $\mathbf{Y}_f^1 \sim p^1$. The corresponding conditional velocity field is:

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

When $\sigma^2 \rightarrow 0$, this reduces to the straight path velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. Unlike FLOWTIME, the learned neural velocity field ν_θ operates on the entire future trajectory at once. The context encoding remains similar to Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ now processes both past and future covariates. The trajectory-level training objective is:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \|\mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|^2 \quad (13)$$

Here ν_θ is implemented as a sequential neural network, such as a Transformer or RNN, that processes the complete trajectory. Sampling therefore requires solving the trajectory-level ODE:

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

with numerical solvers. The baseline thus produces joint samples from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ without the autoregressive decomposition that defines FLOWTIME.

Implementation Details We used this baseline for the dynamical-system ablation, where the goal is to compare factorization choices under comparable training conditions. In this setup, ν_θ was a 4-layer bi-directional LSTM with 128 hidden units, and all other training parameters matched Section 4.1. Training to convergence took ~ 2 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Key Differences from Autoregressive Variant The baseline is included to make the paper’s structural contribution falsifiable. FLOWTIME factorizes the joint distribution through a Markovian sequence of conditional time-step models, whereas the non-autoregressive baseline treats $\mathbf{Y}_f$ as one high-dimensional random variable. This preserves cross-temporal dependencies directly but requires learning a distribution over $\mathbb{R}^{n \times f}$. The computational and uncertainty trade-offs also differ. The non-autoregressive model processes entire trajectories through the velocity-field network, enabling parallel generation of all future time points while increasing memory requirements. FLOWTIME instead models how prediction uncertainty moves through the chain of conditionals, making error accumulation explicit; the non-autoregressive baseline captures joint uncertainty through trajectory-level sampling but does not separately model that accumulation mechanism. Thus the ablation compares interpretable autoregressive uncertainty propagation and window-based computation against direct full-trajectory modeling with less explicit uncertainty dynamics.

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_l^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
- 3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$
- 5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
- 6: Compute target $\mu(\mathbf{Y}_f^s, s|\mathbf{z})$
- 7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
- 8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ

- 1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
- 2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
- 3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
- 4: Return $\psi(\mathbf{Y}_f, 1)$

C EVALUATION METRICS

This appendix defines the two metrics used to evaluate the paper’s claims about probabilistic quality and predictive accuracy. Given an n -dimensional forecast horizon $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ and either its predictive distribution or samples from the model, we report the Continuous Ranked Probability Score (CRPS) and the Normalized Root Mean Squared Error (NRMSE).

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

CRPS is the metric tied to the uncertainty-quantification contribution. It is a proper scoring rule for probabilistic forecasts; for a univariate random variable X with cumulative distribution function (CDF) F_X and observed value x , it is defined as:

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ is the indicator function. CRPS evaluates the alignment between the full predictive distribution and the realized observation x , so lower values indicate better calibrated and more accurate probabilistic forecasts.

For timeseries forecasts, we compute CRPS at each horizon step and average across the forecast window:

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ is the predicted CDF, or empirical CDF from samples, at horizon k , and $y_{\tau+k}$ is the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

NRMSE is the metric tied to the accuracy component of the contribution. It starts from Root Mean Squared Error (RMSE), a standard point-forecast accuracy measure. For predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ and ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, RMSE is:

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

To compare accuracy across systems and datasets with different scales, we report Normalized RMSE (NRMSE). A common normalization divides RMSE by the standard deviation of the ground-truth

series, σ_y :

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observations. Other applications may normalize by the data range ($\max(y) - \min(y)$), but in all cases lower NRMSE means lower prediction error relative to the variability of the series.